

SQL Server - Basics Practice (100 Q&A)

Includes sample schema and 100 SELECT practice questions with T-SQL answers.

-- SAMPLE SCHEMA (SQL Server T-SQL)

-- Tables: Departments, Employees, Customers, Orders, Products

```
CREATE TABLE Departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100)  
);
```

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    job_title VARCHAR(100),  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    department_id INT NULL,  
    manager_id INT NULL,  
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),  
    FOREIGN KEY (manager_id) REFERENCES Employees(employee_id)  
);
```

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    contact_email VARCHAR(100)  
);
```

```
CREATE TABLE Products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    employee_id INT,  
    order_date DATE,  
    amount DECIMAL(10,2),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),  
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id)  
);
```

-- Insert sample rows (add as needed)

-- INSERT INTO Departments VALUES (10, 'Sales'), (20, 'HR'), (30, 'IT');

-- INSERT INTO Employees VALUES (1, 'John', 'Doe', 'Sales Rep', 45000, '2020-06-15', 10, NULL);

Q1: Show all columns and rows from the Employees table.

A1:

```
SELECT * FROM Employees;
```

Q2: Show first_name, last_name and salary of employees.

A2:
SELECT first_name, last_name, salary FROM Employees;

Q3: Show distinct job titles from Employees.

A3:
SELECT DISTINCT job_title FROM Employees;

Q4: Show first 10 rows from Employees (any order).

A4:
SELECT TOP 10 * FROM Employees;

Q5: Show employees whose salary is greater than 50000.

A5:
SELECT * FROM Employees WHERE salary > 50000;

Q6: Show employees hired after 1-Jan-2020.

A6:
SELECT * FROM Employees WHERE hire_date > '2020-01-01';

Q7: Show employees whose first name starts with 'J'.

A7:
SELECT * FROM Employees WHERE first_name LIKE 'J%';

Q8: Show employees where manager_id is NULL.

A8:
SELECT * FROM Employees WHERE manager_id IS NULL;

Q9: Show employees in department 10.

A9:
SELECT * FROM Employees WHERE department_id = 10;

Q10: Show employees not in department 20.

A10:
SELECT * FROM Employees WHERE department_id <> 20;

Q11: Show employees with salary between 30000 and 60000.

A11:
SELECT * FROM Employees WHERE salary BETWEEN 30000 AND 60000;

Q12: Show employees ordered by salary descending.

A12:
SELECT * FROM Employees ORDER BY salary DESC;

Q13: Show top 5 highest salaries (distinct).

A13:
SELECT DISTINCT TOP 5 salary FROM Employees ORDER BY salary DESC;

Q14: Show full name (first + last) as FullName.

A14:
SELECT first_name + ' ' + last_name AS FullName FROM Employees;

Q15: Show employees where last_name ends with 'son'.

A15:
SELECT * FROM Employees WHERE last_name LIKE '%son';

Q16: Show employees with NULL department_id.

A16:
SELECT * FROM Employees WHERE department_id IS NULL;

Q17: Show the count of employees.

A17:
SELECT COUNT(*) AS total_employees FROM Employees;

Q18: Show average salary of employees.

A18:
SELECT AVG(salary) AS avg_salary FROM Employees;

Q19: Show maximum salary.

A19:
SELECT MAX(salary) AS max_salary FROM Employees;

Q20: Show minimum salary.

A20:
SELECT MIN(salary) AS min_salary FROM Employees;

Q21: Show employees whose salary is NULL or 0.

A21:
SELECT * FROM Employees WHERE salary IS NULL OR salary = 0;

Q22: Show employees hired in the year 2022.

A22:
SELECT * FROM Employees WHERE YEAR(hire_date) = 2022;

Q23: Show employees sorted by hire_date ascending, then salary descending.

A23:
SELECT * FROM Employees ORDER BY hire_date ASC, salary DESC;

Q24: Show distinct department_ids from Employees.

A24:
SELECT DISTINCT department_id FROM Employees;

Q25: Show employees where first_name is exactly 'John'.

A25:
SELECT * FROM Employees WHERE first_name = 'John';

Q26: Show employees excluding those with salary above 80000.

A26:
SELECT * FROM Employees WHERE salary <= 80000 OR salary IS NULL;

Q27: Show employees with last_name length > 5 (using LEN).

A27:
SELECT * FROM Employees WHERE LEN(last_name) > 5;

Q28: Show employees where hire_date is NULL.

A28:
SELECT * FROM Employees WHERE hire_date IS NULL;

Q29: Show first 3 characters of first_name.

A29:
SELECT LEFT(first_name,3) AS fn_prefix FROM Employees;

Q30: Show employees with salary in (30000, 45000, 60000).

A30:
SELECT * FROM Employees WHERE salary IN (30000, 45000, 60000);

Q31: Show employees and replace NULL last_name with 'Unknown'.

A31:
SELECT ISNULL(last_name, 'Unknown') AS last_name FROM Employees;

Q32: Show employees and display 'Experienced' if hire_date < '2015-01-01' else 'New'.

A32:
SELECT first_name, last_name,
CASE WHEN hire_date < '2015-01-01' THEN 'Experienced' ELSE 'New' END AS status
FROM Employees;

Q33: Show employees with salary rounded to nearest integer.

A33:
SELECT first_name, salary, ROUND(salary,0) AS salary_rounded FROM Employees;

Q34: Show employees where first_name contains 'an'.

A34:
SELECT * FROM Employees WHERE first_name LIKE '%an%';

Q35: Show employees and the current date using GETDATE().

A35:
SELECT first_name, GETDATE() AS current_dt FROM Employees;

Q36: Show employees with salary greater than department average (use subquery).

A36:
SELECT e.* FROM Employees e
WHERE e.salary > (
SELECT AVG(salary) FROM Employees WHERE department_id = e.department_id
);

Q37: Show employee count per department (including NULL department).

A37:
SELECT department_id, COUNT(*) AS cnt FROM Employees GROUP BY department_id;

Q38: Show employees whose salary equals the maximum salary.

A38:
SELECT * FROM Employees WHERE salary = (SELECT MAX(salary) FROM Employees);

Q39: Show employees with salaries duplicated (appear more than once).

A39:
SELECT salary, COUNT(*) AS cnt FROM Employees GROUP BY salary HAVING COUNT(*) > 1;

Q40: Show employees and their department_name using JOIN.

A40:
SELECT e.first_name, e.last_name, d.department_name
FROM Employees e
LEFT JOIN Departments d ON e.department_id = d.department_id;

Q41: Show employees with salary formatted to 2 decimals (CONVERT).

A41:
SELECT first_name, CONVERT(DECIMAL(10,2), salary) AS salary2 FROM Employees;

Q42: Show employees where first_name or last_name is NULL.

A42:
SELECT * FROM Employees WHERE first_name IS NULL OR last_name IS NULL;

Q43: Show employees whose hire_date is in the last 365 days.

A43:
SELECT * FROM Employees WHERE hire_date >= DATEADD(day, -365, GETDATE());

Q44: Show employees whose salary is a multiple of 1000.

A44:
SELECT * FROM Employees WHERE salary % 1000 = 0;

Q45: Show concatenated name in format 'Last, First'.

A45:
SELECT last_name + ', ' + first_name AS name FROM Employees;

Q46: Show employees with hire_date between two dates.

A46:
SELECT * FROM Employees WHERE hire_date BETWEEN '2018-01-01' AND '2020-12-31';

Q47: Show employees sorted by last_name then first_name.

A47:
SELECT * FROM Employees ORDER BY last_name, first_name;

Q48: Show the top paid employee per department (using TOP with WITH TIES).

A48:
SELECT TOP 1 WITH TIES department_id, first_name, last_name, salary
FROM Employees
ORDER BY ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC);

Q49: Show employees where salary is not null and greater than 0.

A49:
SELECT * FROM Employees WHERE salary IS NOT NULL AND salary > 0;

Q50: Show employees and add a constant column 'Country' with value 'India'.

A50:
SELECT first_name, last_name, 'India' AS Country FROM Employees;

Q51: Show employees and display salary in thousands (salary/1000).

A51:
SELECT first_name, salary, salary/1000.0 AS salary_k FROM Employees;

Q52: Show employees whose first_name contains either 'a' or 'e'.

A52:
SELECT * FROM Employees WHERE first_name LIKE '%a%' OR first_name LIKE '%e%';

Q53: Show employees where last_name IN ('Sharma','Khan','Smith').

A53:
SELECT * FROM Employees WHERE last_name IN ('Sharma','Khan','Smith');

Q54: Show employees with salary rounded up (CEILING).

A54:
SELECT first_name, salary, CEILING(salary) AS sal_up FROM Employees;

Q55: Show employees and compute years since hire (DATEDIFF in years).

A55:
SELECT first_name, DATEDIFF(year, hire_date, GETDATE()) AS years_with_company FROM
Employees;

Q56: Show employees where first_name LIKE pattern with underscore (single char).

A56:
SELECT * FROM Employees WHERE first_name LIKE '_an%';

Q57: Show number of orders per customer (join Customers and Orders).

A57:
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS orders_count
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;

Q58: Show orders placed in the current year.

A58:
SELECT * FROM Orders WHERE YEAR(order_date) = YEAR(GETDATE());

Q59: Show products with price greater than 100.

A59:

```
SELECT * FROM Products WHERE price > 100;
```

Q60: Show customers whose email ends with 'gmail.com'.

A60:

```
SELECT * FROM Customers WHERE contact_email LIKE '%@gmail.com';
```

Q61: Show employees where salary is between dept min and max (using subqueries).

A61:

```
SELECT * FROM Employees e WHERE e.salary BETWEEN (
    SELECT MIN(salary) FROM Employees WHERE department_id = e.department_id
) AND (
    SELECT MAX(salary) FROM Employees WHERE department_id = e.department_id
);
```

Q62: Show employees with same department as employee_id = 5.

A62:

```
SELECT * FROM Employees WHERE department_id = (
    SELECT department_id FROM Employees WHERE employee_id = 5
);
```

Q63: Show employees where salary is NULL or less than 10000.

A63:

```
SELECT * FROM Employees WHERE salary IS NULL OR salary < 10000;
```

Q64: Show employees with hire_date on the first day of a month.

A64:

```
SELECT * FROM Employees WHERE DAY(hire_date) = 1;
```

Q65: Show employees with salary in top 10 percent (approx) using NTILE.

A65:

```
WITH t AS (
    SELECT *, NTILE(10) OVER (ORDER BY salary DESC) AS decile FROM Employees
)
SELECT * FROM t WHERE decile = 1;
```

Q66: Show employees and pad employee_id to 6 digits (FORMAT).

A66:

```
SELECT FORMAT(employee_id, '000000') AS emp_code, first_name FROM Employees;
```

Q67: Show employees whose hire_date equals their manager's hire_date.

A67:

```
SELECT e.* FROM Employees e JOIN Employees m ON e.manager_id = m.employee_id WHERE
e.hire_date = m.hire_date;
```

Q68: Show employees where job_title IS NOT NULL.

A68:

```
SELECT * FROM Employees WHERE job_title IS NOT NULL;
```

Q69: Show employees and the position of 'a' in first_name (CHARINDEX).

A69:

```
SELECT first_name, CHARINDEX('a', first_name) AS pos_a FROM Employees;
```

Q70: Show customers who have not placed any orders.

A70:

```
SELECT c.* FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id WHERE
o.order_id IS NULL;
```

Q71: Show products sorted by price descending and product_name ascending.

A71:

```
SELECT * FROM Products ORDER BY price DESC, product_name ASC;
```

Q72: Show employees and their manager's full name.

A72:

```
SELECT e.first_name + ' ' + e.last_name AS Employee, m.first_name + ' ' + m.last_name AS  
Manager  
FROM Employees e LEFT JOIN Employees m ON e.manager_id = m.employee_id;
```

Q73: Show employees whose salary is between the 25th and 75th percentile (approx using PERCENT_RANK).

A73:

```
WITH p AS (  
    SELECT *, PERCENT_RANK() OVER (ORDER BY salary) AS pr FROM Employees  
)  
SELECT * FROM p WHERE pr BETWEEN 0.25 AND 0.75;
```

Q74: Show orders where amount is greater than average order amount.

A74:

```
SELECT * FROM Orders WHERE amount > (SELECT AVG(amount) FROM Orders);
```

Q75: Show employees and flag if salary > 50000 as 'High' else 'Normal'.

A75:

```
SELECT first_name, salary, CASE WHEN salary > 50000 THEN 'High' ELSE 'Normal' END AS  
sal_flag FROM Employees;
```

Q76: Show employees and their hire month name.

A76:

```
SELECT first_name, DATENAME(month, hire_date) AS hire_month FROM Employees;
```

Q77: Show employees where last_name like 'Mc%'.

A77:

```
SELECT * FROM Employees WHERE last_name LIKE 'Mc%';
```

Q78: Show first 3 characters of product_name.

A78:

```
SELECT LEFT(product_name, 3) AS pfx FROM Products;
```

Q79: Show employees and convert hire_date to varchar YYYY-MM-DD.

A79:

```
SELECT first_name, CONVERT(VARCHAR(10), hire_date, 120) AS hire_ymd FROM Employees;
```

Q80: Show employees where salary multiplied by 1.1 (10% raise) is greater than 60000.

A80:

```
SELECT * FROM Employees WHERE salary * 1.1 > 60000;
```

Q81: Show customers and the length of their name.

A81:

```
SELECT customer_name, LEN(customer_name) AS name_len FROM Customers;
```

Q82: Show orders with order_date weekday number (1-7).

A82:

```
SELECT *, DATEPART(weekday, order_date) AS weekday_no FROM Orders;
```

Q83: Show employees whose salary is in top 3 (overall).

A83:

```
SELECT TOP 3 * FROM Employees ORDER BY salary DESC;
```

Q84: Show products with price formatted to currency (CONCAT).

A84:

```
SELECT product_name, '$' + CONVERT(VARCHAR(20), price) AS price_str FROM Products;
```

Q85: Show employees with duplicate first and last name pairs.

A85:

```
SELECT first_name, last_name, COUNT(*) AS cnt FROM Employees GROUP BY first_name, last_name HAVING COUNT(*) > 1;
```

Q86: Show employees where last_name contains non-letter characters (example using LIKE).

A86:

```
SELECT * FROM Employees WHERE last_name LIKE '%[^A-Za-z]%';
```

Q87: Show employees and their salary difference from company average.

A87:

```
SELECT e.first_name, e.salary, e.salary - (SELECT AVG(salary) FROM Employees) AS diff_from_avg FROM Employees e;
```

Q88: Show employees who joined on Feb 29 (leap day).

A88:

```
SELECT * FROM Employees WHERE MONTH(hire_date) = 2 AND DAY(hire_date) = 29;
```

Q89: Show top 1 employee per department by salary using ROW_NUMBER.

A89:

```
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn FROM Employees
)
SELECT * FROM ranked WHERE rn = 1;
```

Q90: Show employees whose names (first+last) length > 15.

A90:

```
SELECT * FROM Employees WHERE LEN(first_name + last_name) > 15;
```

Q91: Show employees and order count handled by them (join Orders).

A91:

```
SELECT e.employee_id, e.first_name, COUNT(o.order_id) AS orders_count
FROM Employees e LEFT JOIN Orders o ON e.employee_id = o.employee_id
GROUP BY e.employee_id, e.first_name;
```

Q92: Show employees whose first_name contains two consecutive vowels (using LIKE).

A92:

```
SELECT * FROM Employees WHERE first_name LIKE '%aa%' OR first_name LIKE '%ee%' OR first_name LIKE '%ii%' OR first_name LIKE '%oo%' OR first_name LIKE '%uu%';
```

Q93: Show orders with amounts rounded to nearest integer.

A93:

```
SELECT order_id, ROUND(amount,0) AS amount_rounded FROM Orders;
```

Q94: Show employees and a boolean flag (1/0) if salary > 40000.

A94:

```
SELECT first_name, salary, CASE WHEN salary > 40000 THEN 1 ELSE 0 END AS is_over_40k FROM Employees;
```

Q95: Show products with price between 10 and 50 and order by name.

A95:

```
SELECT * FROM Products WHERE price BETWEEN 10 AND 50 ORDER BY product_name;
```

Q96: Show employees whose manager is in a different department.

A96:

```
SELECT e.* FROM Employees e JOIN Employees m ON e.manager_id = m.employee_id WHERE ISNULL(e.department_id, -1) <> ISNULL(m.department_id, -1);
```


Q97: Show employees with salary expressed as percentage of max salary.

A97:

```
SELECT first_name, salary, (salary / (SELECT MAX(salary) FROM Employees) * 100.0) AS  
pct_of_max FROM Employees;
```

Q98: Show customers and the date of their first order.

A98:

```
SELECT c.customer_id, c.customer_name, MIN(o.order_date) AS first_order_date  
FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.customer_name;
```

Q99: Show products that have never been ordered.

A99:

```
SELECT p.* FROM Products p LEFT JOIN Orders o ON p.product_id = o.order_id WHERE  
o.order_id IS NULL;
```

Q100: Show employees and truncate salary to integer part (CONVERT to INT).

A100:

```
SELECT first_name, CONVERT(INT, salary) AS salary_int FROM Employees;
```

